Non-Pipeline SoC CPI Study

Daniel

2026-03-20

Contents

CPI Study - Non-Pipeline Processor

Repo snapshot: PipelineFinal/processor_non_pipeline

Executive summary

- **CPI stays near 1.0** in all three scenarios because the core is not pipelined.
- **CPI rises slightly** only when _rdy/_mem_rdy/_io_rdy dip (memory or IO waits), not due to data/control hazards.
- **Screenshots use fixed windows** (55-200 ns and 500-1200 ns) for consistent visual comparison.

1) Goal and scope

Goal: measure CPI for the non-pipelined processor in three scenarios and collect wave screenshots to compare with pipelined results.

Scenarios: - **No-hazard** (independent instructions, no branches) - **Mixed** (normal blend) - **Hazard-only** (RAW, load-use, branches)

CPI definition: $CPI = \frac{\text{total cycles}}{\text{total instructions}}$

2) Baseline behavior (non-pipeline)

- One instruction at a time; no overlap between stages.
- RAW/load-use/control hazards do not create pipeline bubbles.
- Stalls only come from memory/MMIO ready handshakes.

3) Concepts and signals (quick reference)

Key concepts: - CPI: cycles per instruction, measured as total cycles divided by retired instructions. - Retired instruction: counted when _PC changes while _insn_ce is high. - Ready/handshake: _rdy, _mem_rdy, _io_rdy indicate when the core can progress; dips stretch CPI.

Signals used in waveforms: - _PC: current program counter (instruction address). - _insn_q: current fetched instruction word. - _insn_ce: instruction enable (valid/retire qualifier). - _rdy, _mem_rdy, _io_rdy: overall ready, memory ready, and IO ready.

4) Programs used

4.1 No-hazard program

File: - PipelineFinal/processor_non_pipeline/assembly/input_no_hazards.asm

Notes: - Independent register writes only. - Final infinite loop keeps the sim running; exclude the loop from CPI window if you want strictly no control hazards.

4.2 Mixed program

File: - PipelineFinal/processor_non_pipeline/assembly/input_mixed.asm

Notes: - Includes independent ops, a load with a gap before use, and simple control flow.

4.3 Hazard-only program

File: - PipelineFinal/processor_non_pipeline/assembly/input_hazards.asm

Notes: - Includes RAW, load-use, branch taken, jump, and a loop. - Good for showing control-flow changes and dependency-heavy sequences.

4.4 ABI and assembler details

ABI file: - PipelineFinal/processor_non_pipeline/tools/abi.inc

Key ABI points used by the programs: - Register aliases: a0/a1/a2, v0/v1, t0-t3, s0-s3, fp, sp, lr, gp. - Macro helpers used or available: J (absolute jump), CALL/RET, MOV, SUBI, OR, etc. - The OR macro uses t0 as scratch; avoid relying on t0 across OR unless you re-load it.

All three programs include abi.inc and use the same reset vector (0x0100).

Each program disables Timer1 at start to avoid IRQ noise during CPI counting: - IMM #0x408 - SW zero, zero, #0

5) Measurement method (repeatable)

Vivado setup (first time per project): 1) Open project: PipelineFinal/processor_non_pipeline/processor.xpr

2) Set simulation top to tb_Soc:
   - In Sources, right-click sim/tb_Soc.v -> Set as Top
   - Run Behavioral Simulation
3) Warnings about missing board parts and missing IP folders are expected in this copy and do not block simulation.

Core flow (repeat for each scenario): 1) Assemble the desired program into mem.hex: - Copy the chosen program into assembly/input.asm - Run: python3 tools/assembler.py 2) Run simulation (Vivado or CLI). 3) In the waveform, count: - Total cycles between first and last retired instruction - Total instructions executed 4) Compute CPI = cycles / instructions.

Instruction count method: - Count changes of _PC when _insn_ce is high (each change indicates a committed instruction).

Assembler outputs: - srcs/mem/mem.hex (combined) - srcs/mem/mem_hi.hex - srcs/mem/mem_lo.hex

CLI commands (per scenario):

Step 1: Copy program into input.asm.

```
cp PipelineFinal/processor_non_pipeline/assembly/input_no_hazards.asm \
   PipelineFinal/processor_non_pipeline/assembly/input.asm
```

```
cp PipelineFinal/processor_non_pipeline/assembly/input_mixed.asm \
   PipelineFinal/processor_non_pipeline/assembly/input.asm
```

```
cp PipelineFinal/processor_non_pipeline/assembly/input_hazards.asm \
   PipelineFinal/processor_non_pipeline/assembly/input.asm
```

Step 2: Assemble.

```
python3 tools/assembler.py
```

Step 3: Simulate (XSim).

```
source /Xilinx/2025.1/Vivado/settings64.sh
cd PipelineFinal/processor_non_pipeline/processor.sim/sim_1/behav/ \
   xsim
bash compile.sh
bash elaborate.sh
xsim tb_Soc_behav -key "{Behavioral:sim_1:Functional:tb_Soc}" \
   -tclbatch tb_Soc.tcl \
   -log simulate.log
```

Optional: if using the project venv, run the assembler with:

```
/home/daniel/Desktop/ESRG/2025-2026/2nd Semester/Projects/SoC_and_Peripherals/ \
   .venv/bin/python tools/assembler.py
```

6) Waveform capture checklist

For each scenario: - One screenshot that shows a steady sequence of instructions (baseline flow) - One screenshot that shows any wait/ready stall (if present) - Include time ruler, signal names, and values

Recommended time windows (use the same across scenarios): - Shot A: 55 ns - 200 ns (clean start of execution) - Shot B: 500 ns - 1200 ns (steady mid-run window)

Waveform configs saved: - No-hazard: wavecfgs/tb_Soc_behav_no_hazard.wcfg - Mixed: wavecfgs/tb_Soc_behav_mixed.wcfg - Hazard-only: wavecfgs/tb_Soc_behav_hazard_only.wcfg

Wave DB used by the wave configs: - processor.sim/sim_1/behav/xsim/tb_Soc_behav.wdb

7) Waveform evidence (screenshots)

All screenshots use the same time windows (55-200 ns and 500-1200 ns).

No-hazard

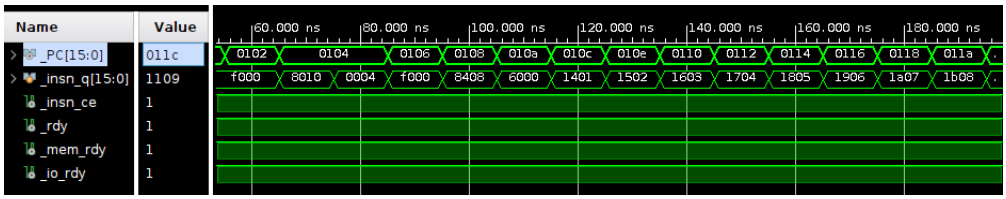

Figure 1: No-hazard Shot A (55-200 ns)

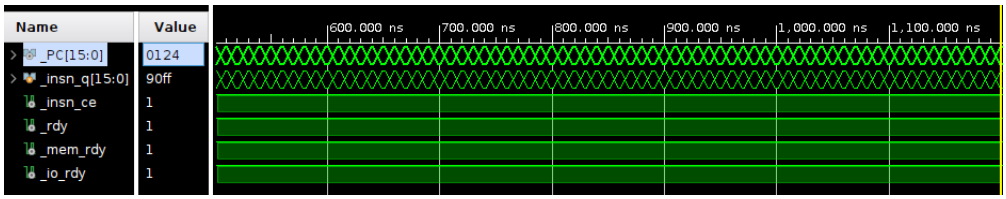

Figure 2: No-hazard Shot B (500-1200 ns)

Mixed

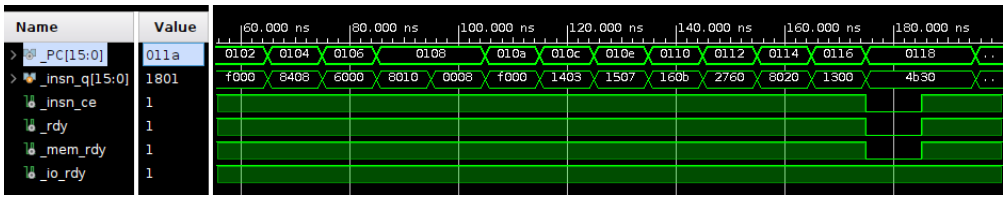

Figure 3: Mixed Shot A (55-200 ns)

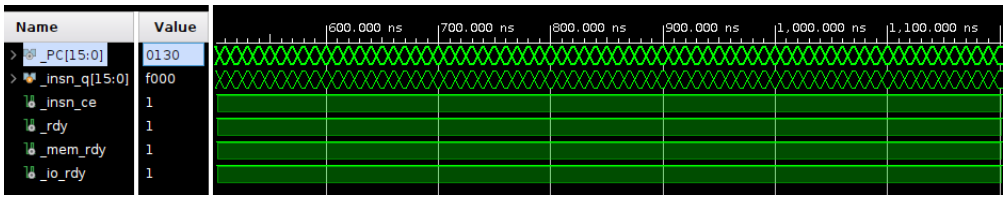

Figure 4: Mixed Shot B (500-1200 ns)

Hazard-only

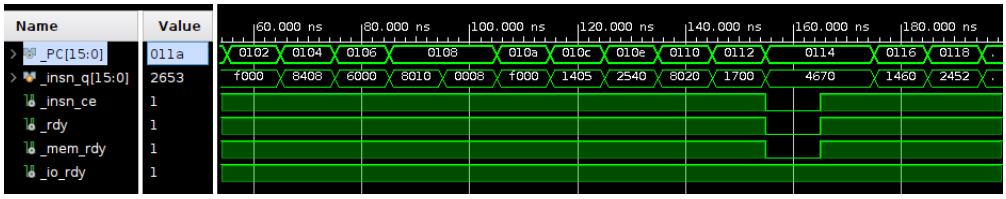

Figure 5: Hazard-only Shot A (55-200 ns)

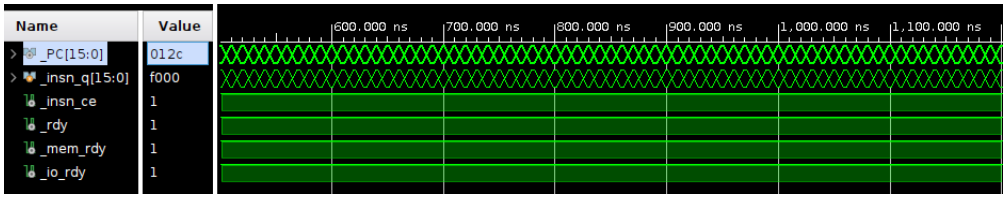

Figure 6: Hazard-only Shot B (500-1200 ns)

8) Step-by-step per scenario

Repeat these steps for each program: no-hazard, mixed, hazard-only.

Scenario A: No-hazard

1. Copy program into input.asm.

   ```
   cp PipelineFinal/processor_non_pipeline/assembly/input_no_hazards.asm \
      PipelineFinal/processor_non_pipeline/assembly/input.asm
   ```

2. Assemble.

   ```
   python3 tools/assembler.py
   ```

3. Run sim with tb_Soc (Behavioral).
4. Screenshot A (baseline flow): _PC, _insn_q, _insn_ce.
5. Screenshot B (if no waits): take another baseline window and note “No waits observed”.
6. Count cycles and instructions; compute CPI.

Scenario B: Mixed

1. Copy program into input.asm.

   ```
   cp PipelineFinal/processor_non_pipeline/assembly/input_mixed.asm \
      PipelineFinal/processor_non_pipeline/assembly/input.asm
   ```

2. Assemble.

   ```
   python3 tools/assembler.py
   ```

3. Run sim with tb_Soc (Behavioral).
4. Screenshot A (baseline flow): _PC, _insn_q, _insn_ce.
5. Screenshot B (if any wait): add _rdy/_mem_rdy/_io_rdy and capture the stall.
6. Count cycles and instructions; compute CPI.

Scenario C: Hazard-only

1. Copy program into input.asm.

   ```
   cp PipelineFinal/processor_non_pipeline/assembly/input_hazards.asm \
      PipelineFinal/processor_non_pipeline/assembly/input.asm
   ```

2. Assemble.

   ```
   python3 tools/assembler.py
   ```

3. Run sim with tb_Soc (Behavioral).
4. Screenshot A (baseline flow): _PC, _insn_q, _insn_ce.

5. Screenshot B (if any wait): add _rdy/_mem_rdy/_io_rdy and capture the stall.
6. Count cycles and instructions; compute CPI.

9) CPI results table (fill in)

Scenario	Total cycles	Instructions	CPI	Load (LW)	Store (SW)	ALU / Logic	Control	Prefix (IMM)	Other (NOP)
No Hazards	145	144	1.01	0.00	5.26	73.68	10.53	10.53	0.00
Mixed	145	140	1.04	8.00	4.00	48.00	20.00	16.00	4.00
Hazards	145	141	1.03	4.55	4.55	50.00	18.18	18.18	4.55

Notes: - Instruction mix uses the same CPI window for counting. - Mixed and hazard-only runs show brief ready dips; no-hazard stays high on _rdy/_mem_rdy/_io_rdy in the same window. - CPI values are taken from the first and last retired instruction between 55 ns and 1495 ns (PC changes with _insn_ce high).

9.1 Interpretation

- The no-hazard CPI is closest to ideal (1.0) because ready signals remain high throughout the window.
- Mixed and hazard-only show short ready dips, which slightly increase CPI without any pipeline-induced stalls.
- Instruction counts differ across scenarios because the programs intentionally use different control flow.

10) Notes to compare with pipeline results

- Expect CPI for non-pipeline to be near 1 unless memory waits are present.
- Pipelined CPI should improve on no-hazard and mixed cases, but may degrade on hazard-only without forwarding.